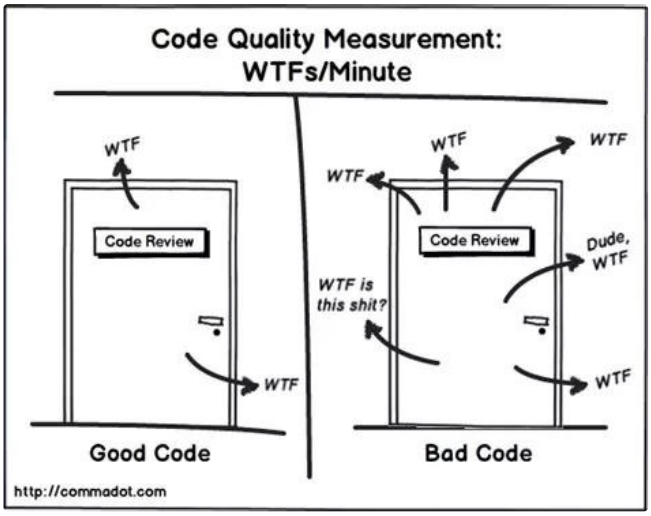
Clean code practices

- Why is Clean Code important?
- Principles of Clean Code
  - Never use magic numbers
  - Meaningful names
  - Small functions are always better
  - DRY principle (Don't Repeat Yourself)
  - Usage of external libraries
  - Sparingly with comments
  - Remove commented-out Code
  - Writing Tests with F.I.R.S.T principal

Clean code is a practice for software development which leads to gain for the developed code: stability, readability and maintainability. The practices were introduced to the public from the developer Robert C. Martin which he tells in his [book](#).

> *Code is clean if it can be understood easily – by everyone on the team. Clean code can be read and enhanced by a developer other than its original author. With understandability comes readability, changeability, extensibility and maintainability.* (Robert C. Martin)



Why is Clean Code important?

Readability and maintenance	Clean code helps to gain clarity, readability and understanding of the code. Readable and understandable code helps on understanding code segments and to fasten the development process.
Team collaboration	Clean and consistent coding standards simplify the communication and collaboration with team members. By following decided coding standards in the team and writing readable code, developers can easily understand each other and work more efficiently.
Debugging and issue fixing	Clean code is designed for clarity and simplicity. Developers can locate and understand easier specific code sections, which leads to identify and resolve issues easier.
Improved quality and reliability	Clean code practices set the focus on established coding standards and writing well-structured code. This reduces the risk of introducing errors and leading to higher quality and more reliable software.

Principles of Clean Code

Never use magic numbers

Using hard-coded numbers in code can cause various issues. These numbers are often referred to as "magic numbers" in programming. Magic numbers are hard-coded values that lack context or meaning, making the code difficult to read and understand. Developers may struggle to grasp the purpose of these numbers, leading to confusion.

Having many magic numbers in your code can complicate refactoring and bug fixing, as changes may need to be made in multiple places. To avoid these problems, it's better to store such numbers in constants with meaningful names. This practice enhances code readability and maintainability by providing clear context and reducing the risk of errors.

Magic number example:

```
public double getTotalCosts(double itemCost, double quantity) {
  return itemCost * quantity * 0.2;
}
```

Correct clean code usage

```
public double getTotalCosts(double itemCost, double quantity) {
  final int taxRate = 0.2;
  return itemCost * quantity * taxRate;
}
```

Meaningful names

Names for variables, functions and classes must reflect their proposes and behavior. Self-explaining names, which make the code self-documented and easier to understand without extra comments. Choosing a meaningful name can take some time, but it will save much more time afterwards. (Chapter 2, page 17)

> A name should tell you why it exists, what it does, and how it is used. If a name requires a comment, then the name does not reveal its intent (Robert C. Martin)

Example: https://gitlab.devops.esolution.services/microservices/sm30/subscription-portal-frontend/-/merge_requests/135

Small functions are always better

Sometimes we encounter some large functions in our daily work. The problem of large functions is that these functions are not maintainable and not readable. Functions should follow the SRP principle (Single responsibility principle), which means one function should follow one purpose and perform it effectively.

If we encounter too big functions that perform too many tasks, one approach would be to break them into smaller, reusable and maintainable functions.

Pseudocode example:

```
public void processData(Object object) {
  ... validateUser ...
  ... calculateValues ...
```

```
  ... format output ...
}
```

Refactored pseudocode example:

```
public boolean isValidUser(Object object) {
  ... user validation ...
}

public double calculateValues(Object object) {
  ... calculate values ...
}

public void showFormattedOutput(Object object) {
  ... show formatted output ...
}
```

The splitting up in much more distinct functions leads to much more readable, maintainable and testable code. If there are changes or fixes to make to the divided functions, it will be easier to understand, identify and modify these functions.

> Functions should do one thing. They should do it well. They should do it only. (Robert C. Martin)

DRY principle (Don't Repeat Yourself)

We should avoid to re-invent the wheel the whole time. This means we should avoid writing the same code logic more than once again. Rewriting or copying the same code every time leads to big overwhelm of maintainability. Instead, we can reuse the implemented logic by using functions, classes, modules or other type of abstractions. This also reduce the risk of errors and bugs as only is needed to change the code only one place if's needed to be changed or adapted.

Usage of external libraries

As developers, we often need to use external libraries to avoid reinventing the wheel. However, sometimes these libraries may no longer be updated or maintained, requiring us to replace them. Directly using library code in business logic can lead to extensive refactoring and maintenance challenges.

To prevent this, we can use wrappers in our projects. Wrapper classes or components, whether for backend or frontend work, can significantly improve the maintainability of our code. Instead of embedding external library code directly into business logic, we can implement it within wrappers and use these wrappers in our project code. This approach ensures that if we need to replace the library in the future, we only need to update the wrapper, leaving the rest of the code unaffected.

Sparingly with comments

The best documentation of the code, it's the code itself. Obvious things are not needed to be commented, and excessive or outdated commentary can lead to confusion. Sometimes there are specific situations in which we need to explain unusual behavior and potential pitfalls, and we need to explain why we do it in this way. Example: https://gitlab.devops.esolution.services/microservices/sm30/subscription-portal-frontend/-/merge_requests/115/diffs?commit_id=a5d22cb62bc572a55a2cc147c1fcb9e9122fa223

Remove commented-out Code

Commented-out code can cause developers to overthink its purpose. Some might assume there's a significant reason why the code wasn't removed, leading to confusion. Additionally, developers may hesitate to delete commented-out code, leaving it in the codebase for too long.

Nowadays, we use version control tools like Git, which ensure that no code is ever truly lost. If we need the code again, we can always retrieve it from the version history. Therefore, the best practice when encountering commented-out code is simple: just delete it!

Writing Tests with F.I.R.S.T principal

Clean tests follow 5 rules. The abbreviation of the rule is **F.I.R.S.T.** Relying on the Clean book of Robert C. Martin (Chapter 9, page 132), the five rules means:

Fast	Tests should be fast. If a test takes too long, we are likely to skip it.
Independent	Tests should be independent of each other. This means that a previous test shouldn't need to be executed for another test to run. For example, in end-to-end (e2e) tests, the mobile view of a feature should run independently from the desktop test, even if the desktop test fails.
Repeatable	Tests should be runnable in every environment. If the tests are repeatable at every stage, we won't have an excuse for why they fail.

	Only e2e tests shouldn’t be executed on prod.
Self-Validating	Tests should have boolean output for validation. If a test passes, it shouldn't require manual validation.
Timely	Tests should be written before production. If we write tests after the production code, we might find that the code is hard to test or not designed to be testable.

Example: https://gitlab.devops.esolution.services/microservices/sm30/subscription-portal-frontend/-/merge_requests/123/diffs?commit_id=c3614da4e66650b47e0b4e4c0bc0a43afbae6b51